Eloquent ORM

简介
基本用法
集体赋值
插入、更新、删除
软删除
时间戳
查询范围
关系
查询关系
预先加载
插入相关模型
触发父模型时间戳
与数据透视表工作
集合
访问器和调整器
日期调整器
模型事件
模型观察者
转为数组或JSON

简介

Laravel 自带的 Eloquent ORM 为您的数据库提供了一个优雅的、简单的 ActiveRecord 实现。每一个数据库的表有一个对应的 "Model" 用来与这张表交互。

在开始之前，确认已在 `app/config/database.php` 文件中配置好数据库连接。

基本用法

首先，创建一个 Eloquent 模型。模型通常在 `app/models` 目录，但是您可以自由地把它们放在任何地方，只要它能根据您的 `composer.json` 文件自动加载。

定义一个 Eloquent 模型

```
class User extends Eloquent {}
```

注意我们并没有告诉 Eloquent 我们为 `User` 模型使用了哪一张表。类名的小写、复数的形式将作为表名，除非它被显式地指定。所以，在这种情况下，Eloquent 将假设 `User` 模型在 `users` 表中保存记录。您可以在模型中定义一个 `table` 属性来指定一个自定义的表名：

```
class User extends Eloquent {

    protected $table = 'my_users';

}
```

> 注意：Eloquent 将假设每张表有一个名为 `id` 的主键。您可以定义 `primaryKey` 属性来覆盖这个约定。同样，您可以定义一个 `connection` 属性来覆盖在使用这个模型时所用的数据库连接。

一旦模型被定义，您可以开始在表中检索和创建记录。注意在默认情况下您将需要在表中定义 `updated_at` 和 `created_at` 字段。如果您不希望这些列被自动维护，在模型中设置 `$timestamps` 属性为 `false`。

获取所有记录

```
$users = User::all();
```

根据主键获取一条记录

```
$user = User::find(1);

var_dump($user->name);
```

> 注意：所有在［查询构建器］中适用的函数在 Eloquent 模型的查询中同样适用。

根据主键获取一条记录或者抛出一个异常

有时您可能希望当记录没有被找到时抛出一个异常，允许您使用 `App::error` 处理器捕捉这些异常并显示404页面。

```
$model = User::findOrFail(1);

$model = User::where('votes', '>', 100)->firstOrFail();
```

注册错误处理器，请监听 `ModelNotFoundException`：

```
use Illuminate\Database\Eloquent\ModelNotFoundException;

App::error(function(ModelNotFoundException $e)
{
    return Response::make('Not Found', 404);
});
```

使用 Eloquent 模型查询

```
$users = User::where('votes', '>', 100)->take(10)->get();

foreach ($users as $user)
{
    var_dump($user->name);
}
```

当然，您也可以使用查询构建器的统计函数。

Eloquent 统计

```
$count = User::where('votes', '>', 100)->count();
```

如果您无法通过通过连贯的接口产生查询，可以使用 `whereRaw`：

```
$users = User::whereRaw('age > ? and votes = 100', array(25))->get();
```

Chunking Results

If you need to process a lot (thousands) of Eloquent records, using the `chunk` command will allow you to do without eating all of your RAM:

```
User::chunk(200, function($users)
{
```

```
    foreach ($users as $user)
    {
        //
    }
});
```

The first argument passed to the method is the number of records you wish to receive per "chunk". The Closure passed as the second argument will be called for each chunk that is pulled from the database.

指定查询的数据库连接

您可能需要在运行一个 Eloquent 查询的时候指定数据库连接，只需要使用 `on` 函数：

```
$user = User::on('connection-name')->find(1);
```

集体赋值

当创建一个新的模型，您可以传递属性的数组到模型的构造函数。这些属性将通过集体赋值分配给模型。这是很方便的，但把用户的输入盲目地传给模型可能是一个严重的安全问题。如果把用户输入盲目地传递给模型，用户可以自由地修改任何或者全部模型的属性。基于这个原因，默认情况下所有 Eloquent 模型将防止集体赋值。

首先，在模型中设置 `fillable` 或 `guarded` 属性。

`fillable` 属性指定哪些属性可以被集体赋值。这可以在类或接口层设置。

在模型中定义 Fillable 属性

```
class User extends Eloquent {

    protected $fillable = array('first_name', 'last_name', 'email');

}
```

在这个例子中，只有三个被列出的属性可以被集体赋值。

`fillable` 的反义词是 `guarded`，将做为一个黑名单而不是白名单：

在模型中定义 Guarded 属性

```
class User extends Eloquent {

    protected $guarded = array('id', 'password');

}
```

在这个例子中，`id` 和 `password` 属性将不被允许集体赋值。所有其他属性将被允许集体赋值。您可以使用 guard 方法阻止所有属性被集体赋值：

阻止所有属性集体赋值

```
protected $guarded = array('*');
```

插入、更新、删除

为了从模型中向数据库中创建一个新的记录，简单地创建一个模型实例并调用 `save` 函数。

保存一个新的模型

```
$user = new User;

$user->name = 'John';

$user->save();
```

> 注意：通常您的 Eloquent 模型将有自动递增的键。然而，如果您希望指定您自定义的键，在模型中设置 `incrementing` 属性为 `false`。

您也可以使用 `create` 函数在一行代码中保存一个新的模型。被插入的模型实例将从函数中返回。但是，在您这样做之前，您需要在模型中指定 `fillable` 或者 `guarded` 属性，因为所有 Eloquent 模型默认阻止集体赋值。

在保存或创建一个使用自增ID的新模型之后，可以通过对象的 `id` 属性获取此自增ID：

```
$insertedId = $user->id;
```

在模型中设置 Guarded 属性

```
class User extends Eloquent {

    protected $guarded = array('id', 'account_id');

}
```

使用模型的 Create 函数

```
// Create a new user in the database...
$user = User::create(array('name' => 'John'));

// Retrieve the user by the attributes, or create it if it doesn't exist...
$user = User::firstOrCreate(array('name' => 'John'));

// Retrieve the user by the attributes, or instantiate a new instance...
$user = User::firstOrNew(array('name' => 'John'));
```

为了更新一个模型，您可以检索它，改变一个属性，然后使用 `save` 函数：

更新一个检索到的模型

```
$user = User::find(1);

$user->email = 'john@foo.com';

$user->save();
```

有时您可能希望不仅保存模型，还有它的所有关系。为此，您可以使用 `push` 函数：

保存一个模型和关系

```
$user->push();
```

您也可以在一组模型上运行更新：

```
$affectedRows = User::where('votes', '>', 100)->update(array('status' => 2));
```

删除一个模型，在实例中调用 `delete` 函数：

删除一个存在的模型

```
$user = User::find(1);

$user->delete();
```

根据主键删除一个模型

```
User::destroy(1);

User::destroy(array(1, 2, 3));

User::destroy(1, 2, 3);
```

当然，您可以在一组模型中运行删除查询：

```
$affectedRows = User::where('votes', '>', 100)->delete();
```

如果您希望简单的在一个模型中更新时间戳，可以使用 `touch` 函数：

只更新模型的时间戳

```
$user->touch();
```

软删除

当软删除一个模型，它并没有真的从数据库中删除。相反，一个 `deleted_at` 时间戳在记录中被设置。为一个模型开启软删除，在模型中指定 `softDelete` 属性：

```
class User extends Eloquent {

    protected $softDelete = true;

}
```

为了在您的表中添加一个 `deleted_at` 字段，您可以在迁移中使用 `softDeletes` 函数：

```
$table->softDeletes();
```

现在，当您在一个模型中调用 `delete` 函数，`deleted_at` 字段将被设置为当前的时间戳。在使用软删除的模型中查询，被软删除的模型将不被包含进查询结果中。为了强制已删除的模型出现在结果集中，在查询中使用 `withTrashed` 函数：

强制软删除的模型到结果集中

```
$users = User::withTrashed()->where('account_id', 1)->get();
```

如果您希望在结果集中只包含软删除的模型，您可以使用 `onlyTrashed` 函数：

```
$users = User::onlyTrashed()->where('account_id', 1)->get();
```

恢复一个已被软删除的记录，使用 `restore` 函数：

```
$user->restore();
```

您也可以在查询中使用 `restore` 函数：

```
User::withTrashed()->where('account_id', 1)->restore();
```

`restore` 函数也可以在关系中被使用：

```
$user->posts()->restore();
```

如果您希望从数据库中真正删除一个模型，您可以使用 `forceDelete` 函数：

```
$user->forceDelete();
```

`forceDelete` 函数也可以在关系中被使用：

```
$user->posts()->forceDelete();
```

检测一个给定的模型实例是否被软删除，可以使用 `trashed` 函数：

```
if ($user->trashed())
{
    //
}
```

时间戳

默认情况下，Eloquent 在数据的表中自动地将维护 `created_at` 和 `updated_at` 字段。只需简单的添加这些 `timestamp` 字段到表中，Eloquent 将为您做剩余的工作。如果您不希望 Eloquent 维护这些字段，在模型中添加以下属性：

禁止自动时间戳

```
class User extends Eloquent {

    protected $table = 'users';

    public $timestamps = false;

}
```

如果您希望定制时间戳的格式，可以在模型中重写 `getDateFormat` 函数：

提供一个定制的时间戳格式

```
class User extends Eloquent {

    protected function getDateFormat()
    {
        return 'U';
    }

}
```

查询范围

范围允许您容易在模型中重用查询逻辑。定义一个范围，简单的用 `scope` 为模型添加前缀：

定义一个查询范围

```
class User extends Eloquent {

    public function scopePopular($query)
    {
        return $query->where('votes', '>', 100);
```

```
    }

    public function scopeWomen($query)
    {
        return $query->whereGender('W');
    }

}
```

使用一个查询范围

```
$users = User::popular()->women()->orderBy('created_at')->get();
```

动态范围

有时您可能希望定义一个接受参数的范围。只需要添加您的参数到您的范围函数：

```
class User extends Eloquent {

    public function scopeOfType($query, $type)
    {
        return $query->whereType($type);
    }

}
```

然后在范围函数调用中传递参数：

```
$users = User::ofType('member')->get();
```

关系

当然，您的数据库可能是彼此相关的。比如，一篇博客文章可能有许多评论，或者一个订单与下订单的用户相关。Eloquent 使得管理和处理这些关系变得简单。Laravel 提供了四种类型的关系：

- 一对一
- 一对多
- 多对多
- Has Many Through
- 多态关系

一对一

一个一对一关系是一种非常基础的关系。比如，一个 `User` 模型可以有一个 Phone`。我们可以在 Eloquent 中定义这个关系：

定义一个一对一关系

```
class User extends Eloquent {

    public function phone()
    {
        return $this->hasOne('Phone');
    }

}
```

传递给 `hasOne` 函数的第一个参数是相关模型的名字。一旦这个关系被定义，我们可以使用 Eloquent 的［动态属性］获取它：

```
$phone = User::find(1)->phone;
```

这条语句所产生的 SQL 语句如下：

```
select * from users where id = 1

select * from phones where user_id = 1
```

注意 Eloquent 假设关系的外键基于模型的名字。在这个例子中假设 `Phone` 模型使用一个 `user_id` 外键。如果您希望覆盖这个惯例，您可以为传递 `hasOne` 函数传递第二个参数。Furthermore, you may pass a third argument to the method to specify which local column that should be used for the association:

```
return $this->hasOne('Phone', 'foreign_key');

return $this->hasOne('Phone', 'foreign_key', 'local_key');
```

在 `Phone` 模型定义逆向关系，我们使用 `belongsTo` 函数：

定义一个逆向关系

```
class Phone extends Eloquent {

    public function user()
    {
        return $this->belongsTo('User');
    }

}
```

在上面的例子中，Eloquent 将在 `phones` 表中寻找 `user_id` 字段。如果您想定义一个不同的外键字段，您可以通过 `belongsTo` 函数的第二个参数传递它：

```
class Phone extends Eloquent {

    public function user()
    {
        return $this->belongsTo('User', 'local_key');
    }

}
```

Additionally, you pass a third parameter which specifies the name of the associated column on the parent table:

```
class Phone extends Eloquent {

    public function user()
    {
        return $this->belongsTo('User', 'local_key', 'parent_key');
    }

}
```

一对多

一个一对多关系的例子是一篇博客文章有许多评论。我们可以像这样定义关系模型：

```
class Post extends Eloquent {
```

```
    public function comments()
    {
        return $this->hasMany('Comment');
    }

}
```

现在我们可以通过 动态属性 访问文章的评论：

```
$comments = Post::find(1)->comments;
```

如果您需要添加进一步的约束检索哪些评论，我们可以调用 `comments` 函数连接链式条件：

```
$comments = Post::find(1)->comments()->where('title', '=', 'foo')->first();
```

再次，如果您想覆盖默认的外键，可以给 `hasMany` 函数传递第二个参数。And, like the `hasOne` relation, the local column may also be specified:

```
return $this->hasMany('Comment', 'foreign_key');

return $this->hasMany('Comment', 'foreign_key', 'local_key');
```

在 `Comment` 模型中定义逆向关系，我们使用 `belongsTo` 函数：

定义逆向关系

```
class Comment extends Eloquent {

    public function post()
    {
        return $this->belongsTo('Post');
    }

}
```

多对多

多对多关系更复杂的关系类型。这种关系的一个例子是一个用户和角色，这个角色也可被其他用户共享。比如，许多用户有 "Admin" 的角色。这个关系需要三个表： `users` ， `roles` 以及

`role_user`。`role_user` 表来源于相关的角色名，并且有 `user_id` 和 `role_id` 字段。

我们可以使用 `belongsToMany` 函数定义一个多对多关系：

```
class User extends Eloquent {

    public function roles()
    {
        return $this->belongsToMany('Role');
    }

}
```

现在，我们可以通过 `User` 模型获取角色：

```
$roles = User::find(1)->roles;
```

如果您想使用一个非常规的表名作为关系表，您可以传递它作为第二个参数给 `belongsToMany` 函数：

```
return $this->belongsToMany('Role', 'user_roles');
```

您也可以覆盖相关的键：

```
return $this->belongsToMany('Role', 'user_roles', 'user_id', 'foo_id');
```

当然，您也可以定义在 `Role` 模型中定义逆向关系：

```
class Role extends Eloquent {

    public function users()
    {
        return $this->belongsToMany('User');
    }

}
```

Has Many Through

The "has many through" relation provides a convenient short-cut for accessing distant relations via an intermediate relation. For example, a `Country` model might have many `Posts` through a `Users` model. The tables for this relationship would look like this:

```
countries
    id - integer
    name - string

users
    id - integer
    country_id - integer
    name - string

posts
    id - integer
    user_id - integer
    title - string
```

Even though the `posts` table does not contain a `country_id` column, the `hasManyThrough` relation will allow us to access a country's posts via `$country->posts`. Let's define the relationship:

```php
class Country extends Eloquent {

    public function posts()
    {
        return $this->hasManyThrough('Post', 'User');
    }

}
```

If you would like to manually specify the keys of the relationship, you may pass them as the third and fourth arguments to the method:

```php
class Country extends Eloquent {

    public function posts()
    {
        return $this->hasManyThrough('Post', 'User', 'country_id', 'user_id');
```

```
    }

}
```

多态关系

多态关系允许在一个关联中一个模型属于多于一个的其他模型。比如，您可能有一个 photo 模型属于一个员工模型或者一个订单模型。我们可以定义像这样定义这个关系：

```
class Photo extends Eloquent {

    public function imageable()
    {
        return $this->morphTo();
    }

}

class Staff extends Eloquent {

    public function photos()
    {
        return $this->morphMany('Photo', 'imageable');
    }

}

class Order extends Eloquent {

    public function photos()
    {
        return $this->morphMany('Photo', 'imageable');
    }

}
```

现在，我们可以为一个员工或者一个订单获取照片：

获取一个多态关系

```
$staff = Staff::find(1);

foreach ($staff->photos as $photo)
{
    //
}
```

然而，"polymorphic" 的真正魔力是当您从 `Photo` 模型中访问员工或者订单的时候：

获取多态关系的属主

```
$photo = Photo::find(1);

$imageable = $photo->imageable;
```

`Photo` 模型的 `imageable` 关系将返回一个 `Staff` 或者 `Order` 实例，依赖于那种类型的模型拥有这个照片。

帮助理解这是怎样工作的，让我们探讨一个多态关系的数据库结构：

多态关系的数据库结构

```
staff
    id - integer
    name - string

orders
    id - integer
    price - integer

photos
    id - integer
    path - string
    imageable_id - integer
    imageable_type - string
```

这里需要注意的键字段是表 `photos` 的 `imageable_id` 和 `imageable_type` 字段。ID 将包含所属员工或订单的 ID，类型将包含所属模型的类名。当访问 `imageable` 关系的时候将允许 ORM 检测所属模型的类型。

查询关系

当为一个模型获取记录的时候，您可能希望基于一个已存在的关系限制结果集的数目。比如，您希望获取至少有一条评论的文章。为此，您可以使用 `has` 函数：

Querying Relations When Selecting

```
$posts = Post::has('comments')->get();
```

您也可以指定一个操作符和数量：

```
$posts = Post::has('comments', '>=', 3)->get();
```

If you need even more power, you may use the `whereHas` and `orWhereHas` methods to put "where" conditions on your `has` queries:

```
$posts = Post::whereHas('comments', function($q)
{
    $q->where('content', 'like', 'foo%');

})->get();
```

动态属性

Eloquent 允许您通过动态属性访问关系。Eloquent 将为您自动加载关系，并且足够聪明到知道是否调用 `get` （为一对多关系）或者 `first` （为一对一关系）。然后，它将通过动态属性访问作为关系。比如，使用下面的 `Phone` 模型：

```
class Phone extends Eloquent {

    public function user()
    {
        return $this->belongsTo('User');
    }

}

$phone = Phone::find(1);
```

不需要像这样打印用户的电子邮件：

```
echo $phone->user()->first()->email;
```

它可以简化为：

```
echo $phone->user->email;
```

> 注意： 返回多个结果的关系，其本质是返回了 `Illuminate\Database\Eloquent\Collection` 类的一个实例。

预先加载

预先加载的存在是为了缓解 N + 1 查询问题。比如，考虑一个 `Author` 相关的 `Book` 模型。关系定义是这样的：

```
class Book extends Eloquent {

    public function author()
    {
        return $this->belongsTo('Author');
    }

}
```

现在，考虑下面的代码：

```
foreach (Book::all() as $book)
{
    echo $book->author->name;
}
```

此循环将执行一个查询获取表中的所有书籍，然后另一个查询为每本书获取作者。所以，如果我们有 25 本书，这个循环将允许 26 个查询。

值得庆幸的是，我们可以使用预先加载来减少查询的数量。这个关系应该被预先加载通过 `with` 函数指定：

```
foreach (Book::with('author')->get() as $book)
{
    echo $book->author->name;
}
```

在上面的循环中，只有两个查询被执行：

```
select * from books

select * from authors where id in (1, 2, 3, 4, 5, ...)
```

明智的使用预先加载可以大大提高应用程序的性能。

当然，您可以一次性预先加载多个关系：

```
$books = Book::with('author', 'publisher')->get();
```

您甚至可以预先加载嵌套关系：

```
$books = Book::with('author.contacts')->get();
```

在上面的例子中，`author` 关系将被预先加载，而且作者的 `contacts` 关系也将被加载。

预先加载约束

有时您可能希望预先加载一个关系，但也为预先加载指定一个条件。这里是一个例子：

```
$users = User::with(array('posts' => function($query)
{
    $query->where('title', 'like', '%first%');
}))->get();
```

在这个例子中，我们预先加载用户的文章，但只限于文章的 title 字段中包含单词 "first" 的文章：

延迟预先加载

从一个已存在的模型中直接预先加载相关的模型也是可能的。这在动态的决定是否加载相关模型或与缓存结合时可能有用。

```
$books = Book::all();

$books->load('author', 'publisher');
```

插入相关模型

您会经常需要插入新的相关模型。比如，你可能希望为一篇文章插入一条新的评论。不需要在模型中手动设置 `post_id` 外键，您可以直接从它的父模型 `Post` 中插入新的评论：

附加一个相关的模型

```
$comment = new Comment(array('message' => 'A new comment.'));

$post = Post::find(1);

$comment = $post->comments()->save($comment);
```

在这个例子中，所插入评论的 `post_id` 字段将自动设置。

相关模型（属于）

当更新一个 `belongsTo` 关系，您可以使用 `associate` 函数，这个函数将为子模型设置外键：

```
$account = Account::find(10);

$user->account()->associate($account);

$user->save();
```

插入相关模型（多对多）

当处理多对多关系时，您也可以插入相关模型。让我们继续使用我们的 `User` 和 `Role` 模型作为例子。我们能够轻松地使用 `attach` 函数为一个用户附加新的角色：

附加多对多模型

```
$user = User::find(1);
```

```
$user->roles()->attach(1);
```

您也可以传递一个属性的数组，存在关系的表中：

```
$user->roles()->attach(1, array('expires' => $expires));
```

当然，`attach` 的反义词是 `detach`：

```
$user->roles()->detach(1);
```

您也可以使用 `sync` 函数附加相关的模型。`sync` 函数接受一个 IDs 数组。当这个操作完成，只有数组中的 IDs 将会为这个模型存在关系表中：

使用 Sync 附加多对多关系

```
$user->roles()->sync(array(1, 2, 3));
```

您也可以用给定的 IDs 关联其他关系表中的值：

同步时添加关系数据

```
$user->roles()->sync(array(1 => array('expires' => true)));
```

有时您可能希望创建一个新的相关的模型，并且在一行中附加它。对于此操作，您可以使用 `save` 函数：

```
$role = new Role(array('name' => 'Editor'));

User::find(1)->roles()->save($role);
```

在这个例子中，新的 `Role` 模型将被保存并附加到用户模型。您也可以传递一个属性的数组加入到在这个操作中所连接的表：

```
User::find(1)->roles()->save($role, array('expires' => $expires));
```

触发父模型时间戳

当一个模型 `belongsTo` 另一个模型，比如一个 `Comment` 属于一个 `Post`，当更新子模型时更新父模型的时间戳通常是有用的。比如，当一个 `Comment` 模型更新，您想自动更新所属 `Post` 的 `updated_at` 时间戳。Eloquent 使之变得容易，只需要在子模型中添加 `touches` 属性包含关系的名字：

```
class Comment extends Eloquent {

    protected $touches = array('post');

    public function post()
    {
        return $this->belongsTo('Post');
    }

}
```

现在，当您更新一个 `Comment`，所属的 `Post` 的 `updated_at` 字段也将被更新：

```
$comment = Comment::find(1);

$comment->text = 'Edit to this comment!';

$comment->save();
```

与数据透视表工作

正如您已经了解到，使用多对多关系需要一个中间表的存在。Eloquent 提供了一些非常有用与这个表交互的方式。比如，让我们假设我们的 `User` 对象有很多相关的 `Role` 对象，在访问这个关系时，我们可以在这些模型中访问数据透视表：

```
$user = User::find(1);

foreach ($user->roles as $role)
{
    echo $role->pivot->created_at;
}
```

注意每一个我们检索的 `Role` 模型将自动赋值给一个 `pivot` 属性。这个属性包含一个代表中间表的模型，并且可以像其他 Eloquent 模型使用。

默认情况下，在 `pivot` 对象中只有键，如果您的数据透视表包含其他的属性，您必须在定义关系时指定它们：

```
return $this->belongsToMany('Role')->withPivot('foo', 'bar');
```

现在，`foo` 和 `bar` 属性将可以通过 `Role` 模型的 `pivot` 对象中访问。

如果您想您的数据透视表有自动维护的 `created_at` 和 `updated_at` 时间戳，在关系定义时使用 `withTimestamps` 方法：

```
return $this->belongsToMany('Role')->withTimestamps();
```

为一个模型在数据透视表中删除所有记录，您可以使用 `detach` 函数：

在一个数据透视表中删除记录

```
User::find(1)->roles()->detach();
```

注意这个操作并不从 `roles` 删除记录，只从数据透视表中删除。

Defining A Custom Pivot Model

Laravel also allows you to define a custom Pivot model. To define a custom model, first create your own "Base" model class that extends `Eloquent`. In your other Eloquent models, extend this custom base model instead of the default `Eloquent` base. In your base model, add the following function that returns an instance of your custom Pivot model:

```
public function newPivot(Model $parent, array $attributes, $table, $exists)
{
    return new YourCustomPivot($parent, $attributes, $table, $exists);
}
```

集合

所有通过 `get` 方法或一个 `relationship` 由Eloquent返回的多结果集都是一个集合对象。这个对象实现了 `IteratorAggregate` PHP 接口，所以可以像数组一样进行遍历。然而，这个对象也有很多其他的函数来处理结果集。

比如，我们可以使用 `contains` 检测一个结果集是否包含指定的主键：

检测一个集合是否包含一个键

```
$roles = User::find(1)->roles;

if ($roles->contains(2))
{
    //
}
```

集合也可以被转换为一个数组或JSON：

```
$roles = User::find(1)->roles->toArray();

$roles = User::find(1)->roles->toJson();
```

如果一个集合被转换为一个字符转，它将以JSON格式返回：

```
$roles = (string) User::find(1)->roles;
```

Eloquent 集合也包含一些方法来遍历和过滤所包含的项：

遍历集合

```
$roles = $user->roles->each(function($role)
{
    //
});
```

过滤集合

过滤集合时，你所提供的回调函数将被作为 array_filter 的回调函数使用。

```
$users = $users->filter(function($user)
{
    if($user->isAdmin())
    {
        return $user;
```

```
    }
});
```

注意： 当过滤集合并转化为JSON格式时，应首先调用 `values` 函数重置数组内所有的键（key）。

对每个集合项应用回调函数

```
$roles = User::find(1)->roles;

$roles->each(function($role)
{
    //
});
```

根据一个值排序集合

```
$roles = $roles->sortBy(function($role)
{
    return $role->created_at;
});
```

有时，您可能希望返回一个自定义的集合以及自己添加的函数。您可以通过在 Eloquent 模型中重写 `newCollection` 函数指定这些：

返回一个自定义集合类型

```
class User extends Eloquent {

    public function newCollection(array $models = array())
    {
        return new CustomCollection($models);
    }

}
```

访问器和调整器

Eloquent 为获取和设置模型的属性提供了一种便利的方式。简单的在模型中定义一个 `getFooAttribute` 函数声明一个访问器。记住，这个函数应该遵循 "camel-casing" 拼写格式，即使您的数据库使用 "snake-case"：

定义一个访问器

```
class User extends Eloquent {

    public function getFirstNameAttribute($value)
    {
        return ucfirst($value);
    }

}
```

在上面的例子中，`first_name` 字段有一个访问器。注意属性的值被传递到访问器。

调整器以类似的方式声明：

定义一个调整器

```
class User extends Eloquent {

    public function setFirstNameAttribute($value)
    {
        $this->attributes['first_name'] = strtolower($value);
    }

}
```

日期调整器

默认情况下，Eloquent 将转换 `created_at`、`updated_at` 以及 `deleted_at` 字段为 Carbon 的实例，它提供了各种有用的函数，并且继承自原生 PHP 的 `DateTime` 类。

您可以指定哪些字段自动调整，设置禁用调整，通过在模型中重写 `getDates` 函数：

```
public function getDates()
{
```

```
    return array('created_at');
}
```

当一个字段被认为是一个日期，您可以设置它的值为一个 UNIX 时间戳，日期字符串（`Y-m-d`），日期时间字符串，当然也可以是一个 `DateTime` 或 `Carbon` 实例。

完全禁用日期调整，从 `getDates` 函数中返回一个空数组：

```
public function getDates()
{
    return array();
}
```

模型事件

Eloquent 模型触发一些事件，允许您使用以下函数在模型的生命周期内的许多地方插入钩子：`creating`、`created`、`updating`、`updated`、`saving`、`saved`、`deleting`、`deleted`、`restoring`、`restored`。

每当一个新的项目第一次被保存，`creating` 和 `created` 事件将被触发。如果一个项目不是新的并且 `save` 函数被调用，`updating`/`updated` 事件将被触发。在这两种情况下，`saving`/`saved` 事件都将被触发。

如果从 `creating`，`updating` 或者 `saving` 返回 `false` ，该操作将被取消：

通过事件取消保存操作

```
User::creating(function($user)
{
    if ( ! $user->isValid()) return false;
});
```

Eloquent 模型也包含一个静态的 `boot` 函数，它可以提供一个方便的地方注册事件绑定。

设置一个模型 Boot 函数

```
class User extends Eloquent {

    public static function boot()
    {
```

```
        parent::boot();

        // Setup event bindings...
    }

}
```

模型观察者

为加强处理模型事件，您可以注册一个模型观察者。一个观察者类可以包含很多函数对应于很多模型事件。比如，`creating`，`updating`，`saving` 函数可以在一个观察者中，除其他模型事件名之外。

因此，比如，一个模型观察者可以像这样：

```
class UserObserver {

    public function saving($model)
    {
        //
    }

    public function saved($model)
    {
        //
    }

}
```

您可以使用 `observe` 函数注册一个观察者实例：

```
User::observe(new UserObserver);
```

转为数组或JSON

当构建JSON APIs，您可能经常需要转换您的模型和关系为数组或JSON字符串。所以，Eloquent 包含这些方法。转换一个模型和它加载的关系为一个数组，您可以使用 `toArray` 函数：

转换一个模型为数组

```
$user = User::with('roles')->first();

return $user->toArray();
```

注意模型的全部集合也可以被转为数组：

```
return User::all()->toArray();
```

转换一个模型为JSON字符串，您可以使用 `toJson` 函数：

转换一个模型为JSON字符串

```
return User::find(1)->toJson();
```

注意当一个模型或集合转换为一个字符串，它将转换为JSON，这意味着您可以直接从应用程序的路由中返回 Eloquent 对象！

从路由中返回一个模型

```
Route::get('users', function()
{
    return User::all();
});
```

有时您可能希望限制包含在模型数组或JSON中的属性，比如密码，为此，在模型中添加一个隐藏属性：

从数组或JSON转换中隐藏属性

```
class User extends Eloquent {

    protected $hidden = array('password');

}
```

> 注意： 如果关系被隐藏了，请使用关系的 method 名称，而不是动态的访问名称。

或者，您可以使用 `visible` 属性定义一个白名单：

```
protected $visible = array('first_name', 'last_name');
```

某些时候，你可能需要向数据库添加一组属性，而这组属性并在数据库中没有对应的字段。为了能够实现这一需求，可以简单的为每个属性定义一个访问器：

```
public function getIsAdminAttribute()
{
    return $this->attributes['admin'] == 'yes';
}
```

一旦创建了访问器，只需将属性添加到模型的`appends`属性中：

```
protected $appends = array('is_admin');
```

一旦将属性添加到`appends`列表中，它就将被包含在模型和JSON表单中。

LARAVEL IS A TRADEMARK OF TAYLOR OTWELL. COPYRIGHT © TAYLOR OTWELL.

DESIGNED BY
JACK McDADE